

# DESENHO DE ALGORITMOS

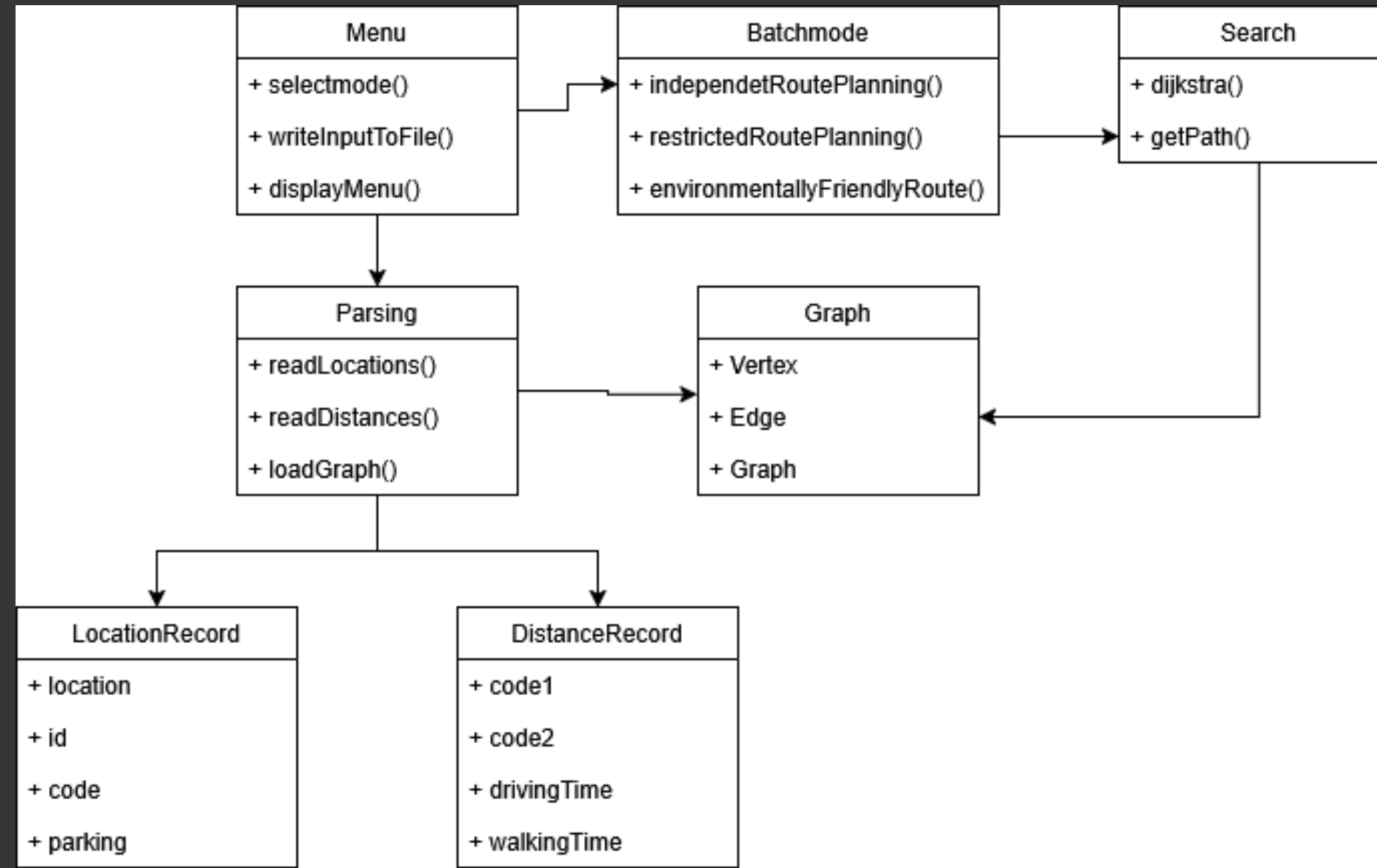
## *Individual Route Planning Tool*

Gabriel Sousa - up202108816

Gustavo Madureira - up202304978

Pedro Alves - up202305444

# CLASS DIAGRAM





# DATASET READING

- `readLocations(string &filename)`

Reads the Locations Dataset and stores them in a vector

- `readDistances(string &filename)`

Reads the Distances Dataset and stores them in a vector



# DATASET READING

- `loadGraph(Graph<int>& graph)`

Creates a map using the location code as the key.

Adds the vertices and bidirectional edges to a graph.



# GRAPH STRUCTURE

## Vertices

Each vertex in the graph corresponds to a location  
Each vertex is also associated to a ignored flag, parking flag, savedPath and savedDist.



```
// auxiliary fields
bool visited = false; // used by DFS, BFS, Prim ...
bool processing = false; // used by isDAG (in addition to the visited attribute)
int low = -1, num = -1; // used by SCC Tarjan
unsigned int indegree; // used by topsort
double dist = 0;
Edge<T> *path = nullptr;
double savedDist = 0; // save current dist and path
Edge<T> *savedPath = nullptr;
bool ignoreFlag = false; // if true then ignore Vertex when searching for path
bool parking=false;
```





# GRAPH STRUCTURE

## Edges

Each edge in the graph corresponds to a distance and connects two vertices.

Each edge is also associated to a walkingTime, drivingTime and ignored flag.



```
Vertex<T> * dest; // destination vertex
int drivingTime;
int walkingTime;

// auxiliary fields
bool selected = false;
bool ignoreFlag = false; // if true then ignore Edge when searching for path

// used for bidirectional edges
Vertex<T> *orig;
Edge<T> *reverse = nullptr;
```

# IMPLEMENTED FUNCTIONALITIES



```
void independentRoutePlanning(Graph<int>& graph);  
  
/**...*/  
void restrictedRoutePlanning(Graph<int>& graph);  
  
/**...*/  
void environmentallyFriendlyRoutePlanning(Graph<int>& graph);
```

independentRoutePlanning -  $O((V+E)\log V)$

restrictedRoutePlanning -  $O(2(V+E)\log V)$  (worstcase)

environmentallyFriendlyRoute -  $O((V+E)\log V)$

---

# INDEPENDENT ROUTE PLANNING

```
// Computations
double firstPathLength = 0, secondPathLength = 0;
Vertex<int>* endVertex = urbanGraph.findVertex(in: endNode);
std::vector<int> firstPath, secondPath;

dijkstra(g: &urbanGraph, origin: startNode);
firstPathLength = endVertex->getDist();
firstPath = getPath(g: &urbanGraph, origin: startNode, dest: endNode);

// Ignore all used Nodes to compute the Alternate Driving Route (except start and end nodes)
for (int i = 1; i < firstPath.size() - 1; ++i) {
    auto vertex : Vertex<int>* = urbanGraph.findVertex(in: firstPath[i]);
    vertex->setIgnoreFlag(ignore: true);
}

dijkstra(g: &urbanGraph, origin: startNode);
secondPathLength = endVertex->getDist();
secondPath = getPath(g: &urbanGraph, origin: startNode, dest: endNode);
```



# RESTRICTED ROUTE PLANNING

```
// Computations
std::vector<int> firstPath, secondPath;
Vertex<int>* endVertex = urbanGraph.findVertex( in: endNode);
Vertex<int>* includeVertex = urbanGraph.findVertex( in: includeNode);
double firstPathLength = 0, secondPathLength = 0;

dijkstra(g: &urbanGraph, origin: startNode);
if (includeNode == -1 || startNode == includeNode || endNode == includeNode) {
    // No IncludeNode or it's the same as startNode/endNode
    firstPathLength = endVertex->getDist();
    firstPath = getPath(g: &urbanGraph, origin: startNode, dest: endNode);

    if (firstPath.empty()) {
        std::cout << "Error: No path found from startNode " << startNode << " to endNode " << endNode << ".\n";
        outFile << "Source: " << startNode << "\n"
            << "Destination: " << endNode << "\n"
            << "RestrictedDrivingRoute: none\n";
        return;
    }
}
```

```
} else {
    // Path from startNode to includeNode
    firstPathLength = includeVertex->getDist();
    firstPath = getPath(g: &urbanGraph, origin: startNode, dest: includeNode);

    if (firstPath.empty()) {
        std::cout << "Error: No path found from startNode " << startNode << " to includeNode " << includeNode << ".\n";
        outFile << "Source: " << startNode << "\n"
            << "Destination: " << endNode << "\n"
            << "RestrictedDrivingRoute: none\n";
        return;
    }

    dijkstra(g: &urbanGraph, origin: includeNode);
    secondPathLength = endVertex->getDist();
    secondPath = getPath(g: &urbanGraph, origin: includeNode, dest: endNode);

    if (secondPath.empty()) {
        std::cout << "Error: No path found from includeNode " << includeNode << " to endNode " << endNode << ".\n";
        outFile << "Source: " << startNode << "\n"
            << "Destination: " << endNode << "\n"
            << "RestrictedDrivingRoute: none\n";
        return;
    }
}
```

# ENVIRONMENTALLY FRIENDLY ROUTE PLANNING

```
// Check constraints
if (urbanGraph.areAdjacent( origin: startNode, dest: endNode)) {
    outFile << "Source: " << startNode << "\nDestination: " << endNode << "\n";
    outFile << "DrivingRoute:none\nParkingNode:none\nWalkingRoute:none\n";
    outFile << "Message: Origin and destination are adjacent, violating constraints.\n";
    return;
}

if(urbanGraph.findVertex( in: startNode)->hasParking() ||urbanGraph.findVertex( in: endNode)->hasParking()){
    outFile << "Source: " << startNode << "\nDestination: " << endNode << "\n";
    outFile << "DrivingRoute:none\nParkingNode:none\nWalkingRoute:none\n";
    outFile << "Message: Origin or destination is a parking node, violating constraints.\n";
    return;
}

// Computations
dijkstra( g: &urbanGraph, origin: startNode); // all driving routes from start node
for (auto node :Vertex<int>* : urbanGraph.getVertexSet()) {
    node->saveDist();
    node->savePath();
}

dijkstra( g: &urbanGraph, origin: endNode, driving: false); // all walking routes from end node

int bestTotalTime = INT_MAX, bestParkingNode = -1;
int firstSuggestedTotalTime = INT_MAX, firstSuggestedParkingNode = -1;
int secondSuggestedTotalTime = INT_MAX, secondSuggestedParkingNode = -1;
for (auto node :Vertex<int>* : urbanGraph.getVertexSet()) {
    if (!node->hasParking()) continue;

    //          driving time          walking time
    int totalTime = node->getSavedDist() + node->getDist();
    if (totalTime < bestTotalTime && node->getDist() <= maxWalkTime) { // obeys maxWalkTime restriction
        bestTotalTime = totalTime;
        bestParkingNode = node->getInfo();
    }
    else if (totalTime < firstSuggestedTotalTime) { // new best suggestion, push the old best suggestion to second place
        secondSuggestedTotalTime = firstSuggestedTotalTime;
        secondSuggestedParkingNode = firstSuggestedParkingNode;
        firstSuggestedTotalTime = totalTime;
        firstSuggestedParkingNode = node->getInfo();
    }
    else if (totalTime < secondSuggestedTotalTime) { // new second best suggestion, remove the old second best suggestion
        secondSuggestedTotalTime = totalTime;
        secondSuggestedParkingNode = node->getInfo();
    }
}
```



# USER INTERFACE



```
===== Route Planning =====  
1. Independent Route Planning  
2. Restricted Route Planning  
3. Environmentally Friendly Route Planning  
4. Exit  
Enter your choice:|
```



# USER INTERFACE



```
===== Mode Selection =====
```

- 1. Manual Input
- 2. Automatic Input
- 3. Back

```
Enter your choice:1
```

```
Source:3
```

```
Destination:8
```

```
Data successfully saved in input.txt!
```

```
Start Location: VIELA MEIRAL
```

```
Destination: VIA NORTE (CIRC.)
```

# HIGHLIGHT FUNCTIONALITY



- All functionalities were implemented
- We added a feature in the menu, that displays the Location name instead of their Id.



# MAIN DIFFICULTIES AND PARTICIPATION

- We don't think there was a noticeable difficulty that we faced while developing this project.
- The main difficulty might be trying to find a better, in terms of time complexity algorithm

Participation:

| up202108816   | up202304978       | up202305444 |
|---------------|-------------------|-------------|
| Gabriel Sousa | Gustavo Madureira | Pedro Alves |
| 40%           | 40%               | 20%         |

# ADDITIONAL INFO

User Interface example continuation

BestDrivingRoute:

1. VIELA MEIRAL
2. LAGARTEIRO
3. BONJÓIA
4. IGREJA DE CAMPANHÃ
5. MONTE CAMPANHÃ
6. BR. FALCÃO
7. CHAVES OLIVEIRA
8. CARTES
9. CONTUMIL (METRO)
10. BR.CONTUMIL
11. FONTE DE CONTUMIL
12. BONFIM BARREIROS
13. R. CONTUMIL

14. BARROCAS

15. DR. EDUARDO SANTOS SILVA
16. IGREJA AREOSA
17. FONTE DO OUTEIRO
18. PAVILHÃO LUÍS FALCÃO
19. FACULDADE DE ENGENHARIA
20. ESC. SUP. EDUCAÇÃO
21. HOSP. S. JOÃO (CIRCUNVALAÇÃO)
22. IPO (CIRCUNVAL.)
23. S. TOMÉ
24. AMIAL
25. CAPUCHINHOS
26. VIA NORTE (CIRC.)

Route time: 67

AlternateDrivingRoute:

1. VIELA MEIRAL
2. MEIRAL
3. S. ROQUE
4. S. ROQUE (CIRCUNV.)
5. FÁBRICA DO COBRE
6. VILA COVA
7. PARQUE NASCENTE
8. RANHA
9. REBORDÃOS
10. VARZIELA
11. AREOSA
12. ENXURREIRAS
13. ASPRELA
14. DR. BARROS
15. OLIVEIRA GAIO

16. MOALDE

17. S. MAMEDE
18. GODINHO FARIA
19. PEDRA VERDE
20. LIDADOR-SOEIRO MENDES
21. HOSP.MAGALHÃES LEMOS
22. PRECIOSA
23. RUELA
24. S.RA PENHA
25. R. DO SENHOR
26. ALTO DO VISO
27. QUARTEL MILITAR
28. BR. STO. EUGÉNIO
29. CONGOSTAS
30. MTE BURGOS (CIRC.)
31. VIA NORTE (CIRC.)

Route time: 159